



Hello! How is it going?



too much sleepy

it is going

muy bien going well

okay amazing goodly

fine good

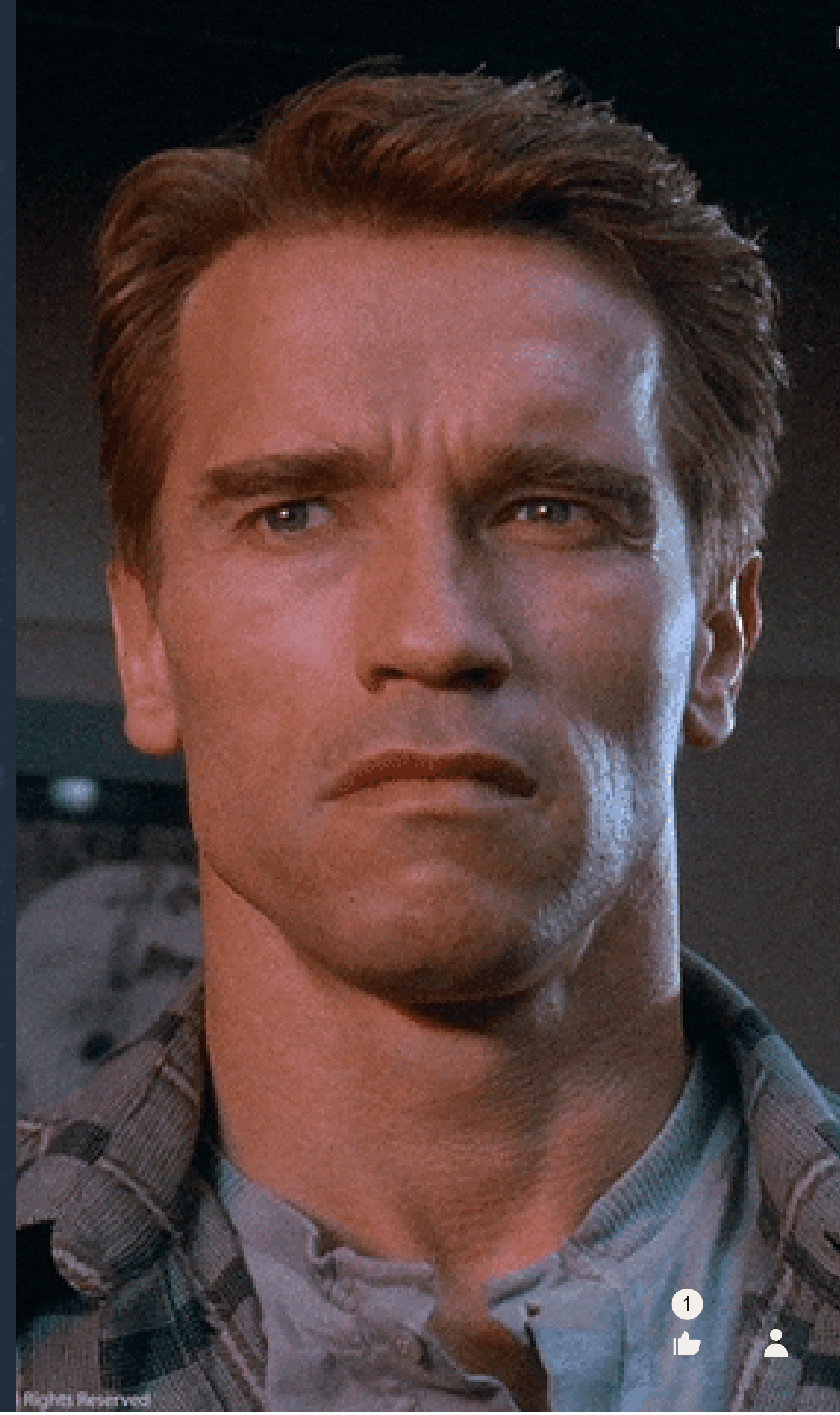
i do not know

wunderbar

Agenda

- Recall last session
- Discuss current topic
- Practice
- Quiz

Total Recall



What do you remember most from few previous workshops? (short answer)

sleep

Threads, wait(), sync,
sleep() etc

the difference between
sleep and wait)

pool

nothing..... my brain is
cleared

```
new Thread() ->
System.out.println("Hello
Thread");
```


How does Thread.sleep() differ from wait()?

0 ✗

Pauses thread permanently

0 ✗

Can wake threads randomly

1 ✗

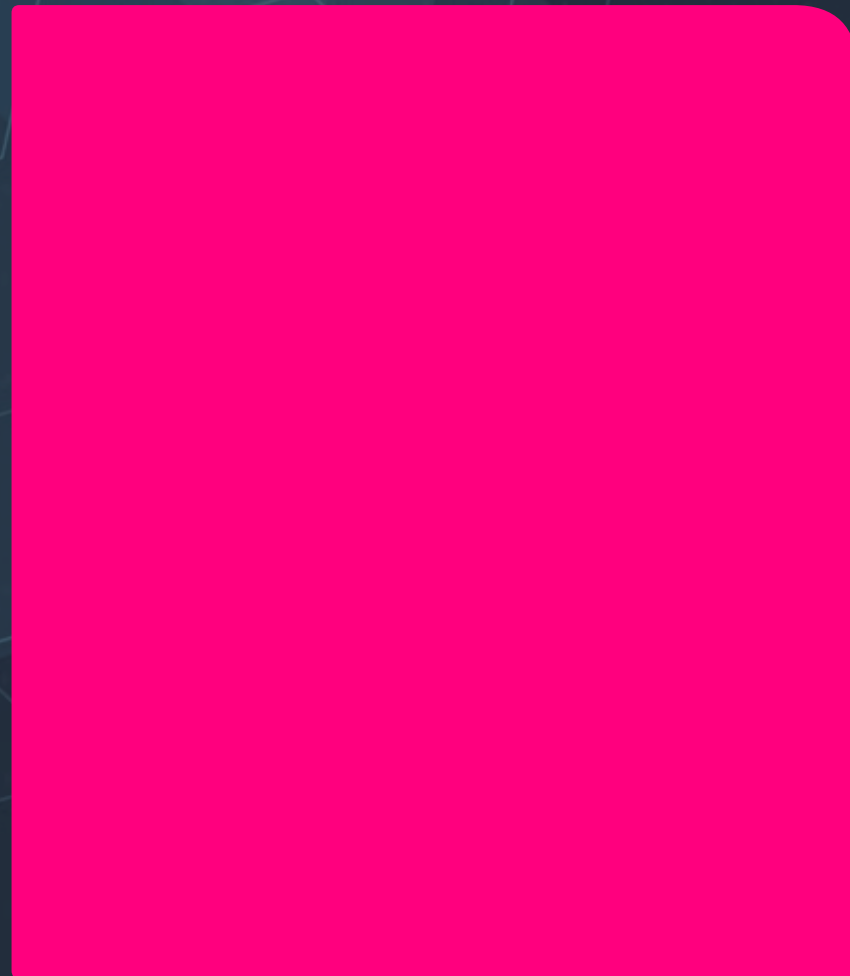
Requires an object monitor

7 ✓

Pauses thread for specified time

How do you apply synchronization?

8 ✓



Use `synchronized` keyword

0 ✗

Override thread joining

0 ✗

Add @Sync annotation

0 ✗

Pass threads through a constructor

What issue does synchronization solve?

8 ✓



0 ✗

Improves thread speed

0 ✗

Handles thread termination

1 ✗

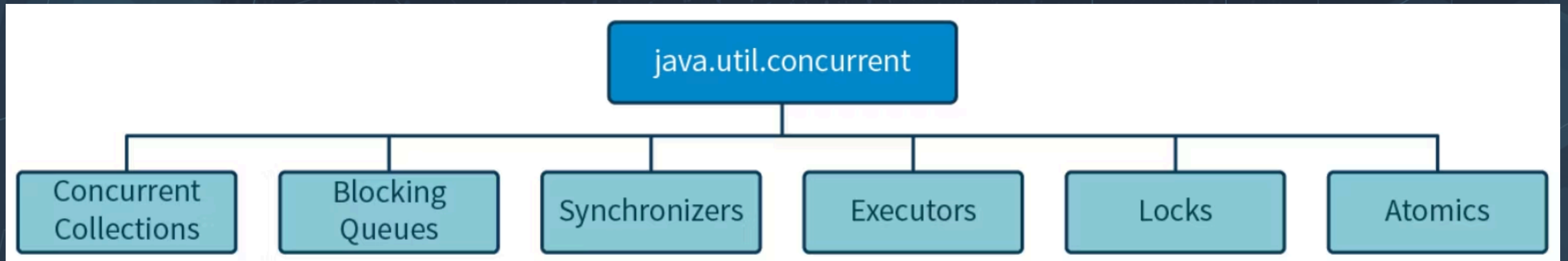
Avoids method overflow

Next topic

Random fact

If you Google the word "askew" the entire page will tilt - as if askewed. Warning: it might make you feel quite sick.

1. Intro to High-Level Concurrency APIs



java.util.concurrent

Concurrent collections are designed to be used in a multithreaded context:

- ConcurrentHashMap
- ConcurrentSkipListMap
- ConcurrentSkipListSet
- CopyOnWriteArrayList
- CopyOnWriteArraySet

When many threads are expected to access a particular collection, it is preferable to use concurrent collections, as they allow multiple threads to read/write simultaneously.

BlockingQueue and BlockingDeque

The blocking queues are of the types BlockingQueue and BlockingDeque. They block a thread requesting an element from an empty queue until an element available for retrieval appears in the queue.

Synchronizers are classes that provide a ready-made threading solution for typical multithreaded tasks:

- Semaphore
- CyclicBarrier
- CountdownLatch
- Exchanger

These synchronizer classes define the interaction between threads in the following cases: data exchange, preparatory/preliminary actions, waiting for an event or state, etc.

Executors define a standardized interface for running and managing threads: a thread pool, asynchronous thread execution, and a concurrent execution framework. Depending on the type of executor, tasks can be executed in a newly created thread or an existing one—sequentially or in parallel.

Executors include:

- The Executor interface and its inheritors—the thread pool
- The Callable interface, which defines a thread with a result return—i.e., asynchronous thread execution
- The Future interface, which receives a result of a thread of the Callable type
- The Fork/Join framework for parallel execution of threads

Locks are interfaces and classes of the `java.util.concurrent.locks` package. They provide frameworks other than basic synchronization and monitors, such as:

- Thread locking
- Waiting for resource access conditions

Atomics

Atomics are classes of the `java.util.concurrent.atomic` package. They maintain thread-oriented programming without blocking specific data types. For example, the `AtomicInteger` or `AtomicLong` classes provide an atomic method for the increment operation.

Atomic classes

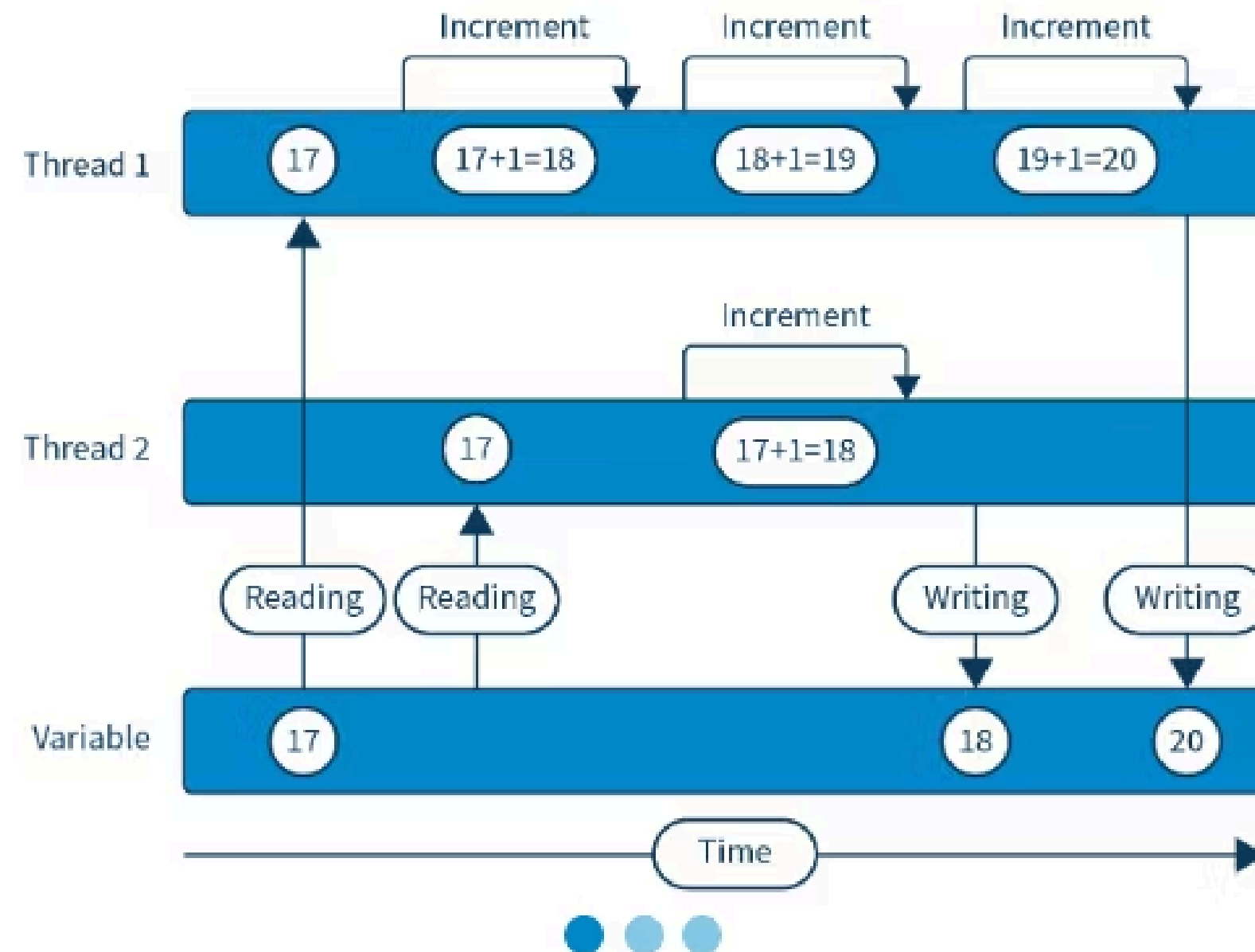
Atomic classes ensure certain operations will be thread-safe (e.g., increment and decrement operations, updating and adding a value, etc.). Atomic classes include AtomicInteger, AtomicBoolean, AtomicLong, AtomicIntegerArray, etc.).

Atomic variables

Atomic variables are variables of atomic classes. They ensure the correct execution of compound/complex operations without external synchronization (e.g., increment/decrement). With volatile variables, such operations are performed non-atomically.

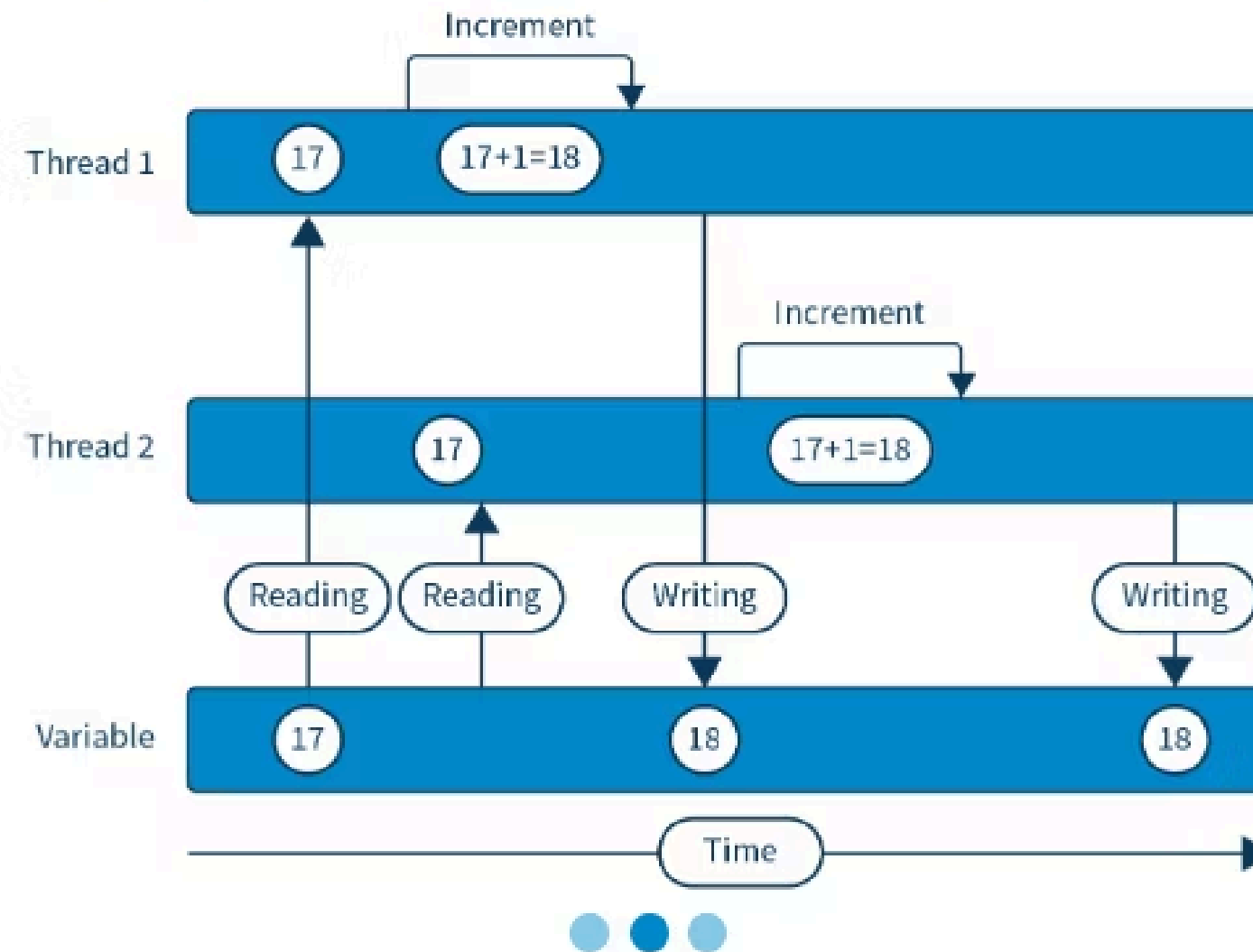
Regular

The `count1` field of the `MyCounter` class is declared a regular instance field. Therefore, threads read its value into their internal memory (stack thread) and update it unpredictably in the main memory—hence the inconsistency.



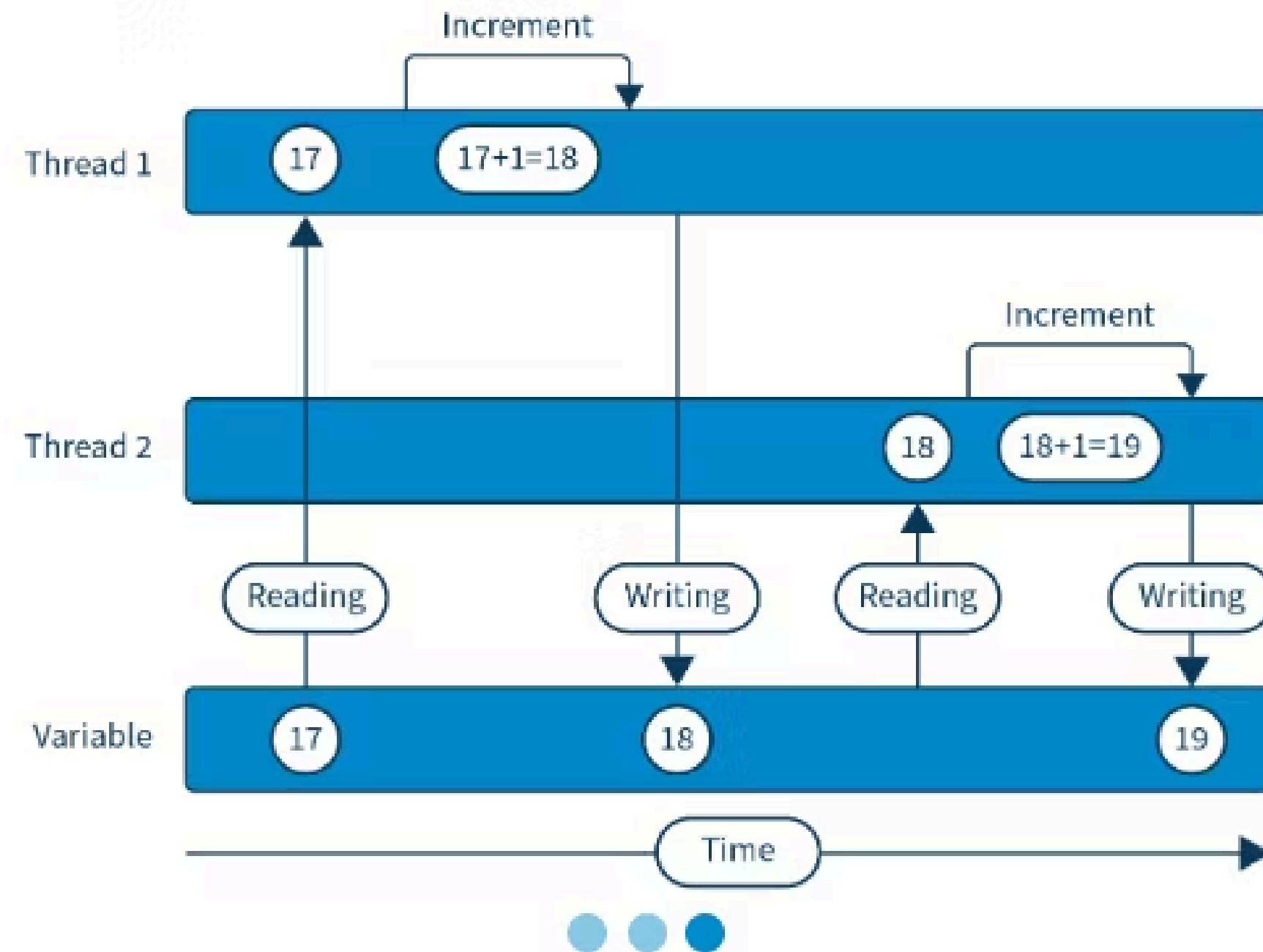
Volatile

The `count2` field of the `MyCounter` class is declared a `volatile` variable. This synchronizes only reading and writing operations of a variable from/to the main memory between threads. However, threads perform a non-atomic increment operation on this field; the value is first read and then modified and assigned as new. In other words, a reading operation is consistent across threads, but an updating operation is not. When one thread begins to change a variable, another thread can simultaneously read the unchanged value. That is, both threads work with the same value at the same time.



Atomic

The `count3` field of the `MyCounter` class is declared as an atomic variable, which means that the increment operation is performed sequentially across threads. The result is expected—the value of the field is 100000000.



BREAK



2. Concurrent Collections

To be explored:

- BlockingQueue
- BlockingDeque
- ConcurrentHashMap
- CopyOnWriteArrayList

The `ArrayBlockingQueue<E>` class has a limited size and is based on arrays. Therefore, when creating an object of this type, you must specify its size.

The `LinkedBlockingQueue<E>` class is potentially borderless and based on linked nodes. However, you can set the size or use the default `Integer.MAX_VALUE`.

The `PriorityBlockingQueue<E>` class is an unbounded heap-based queue that has the same ordering rules as a regular priority queue—the `PriorityQueue` class. This type of queue supplies blocking retrieval operations.

The `DelayQueue<E>` class is an unbounded queue and has the following features:

- Only elements whose delay has elapsed are selected.
- The elements are arranged as the delay expires.
- If no item in the queue has timed out, the queue is blocked for fetching.

implements the `BlockingQueue` or `BlockingDeque` interface

BlockingQueue

BlockingQueue is a powerful interface in the `java.util.concurrent` package designed for **Producer-Consumer** patterns. Its defining feature is that it can **wait**: it waits for the queue to become non-empty when retrieving an element, and waits for space to become available when storing an element.

Comprehensive Example

Since **BlockingQueue** is an interface, this example uses **ArrayBlockingQueue**, which uses a fixed-size array and a single lock to manage traffic.

BlockingDeque

BlockingDeque (Double-Ended Queue) is a sub-interface of **BlockingQueue** that allows for thread-safe, blocking operations at **both ends** of the queue. It is essentially a "Blocking Stack" and "Blocking Queue" combined.

It is most commonly used in **Work-Stealing** algorithms, where a thread usually processes its own tasks from the front (LIFO) but can "steal" tasks from the back of another thread's deque to balance the load.

Comprehensive Example

This example uses **LinkedBlockingDeque**, which is the most common implementation.

The SynchronousQueue Class

The SynchronousQueue class is a type of blocking queue that supports the passing of data between threads in one direction: One thread sends data, and the other receives it.

This queue is special because it has no data storage capacity. Each insert operation must wait for a corresponding delete operation by another thread, and vice versa. The element is only present when you try to remove it.

[Check example](#)

ConcurrentHashMap

`ConcurrentHashMap` provides a thread-safe alternative to `HashMap` by using **fine-grained locking**, which allows multiple threads to read and write simultaneously without locking the entire object. Unlike `Hashtable` or `Collections.synchronizedMap`, it uses **bucket-level locking** (via `synchronized` on node heads) and **CAS (Compare-And-Swap)** operations to ensure high performance in multi-threaded environments.

Comprehensive Example

Next example demonstrates core atomic operations, handling null constraints, and concurrent modification behaviors.

Key Technical Aspects

- **No Global Lock:** While `Hashtable` locks the whole map for every operation, `ConcurrentHashMap` only locks the specific **bucket** being updated.
- **Lock-Free Reads:** The `get()` operation typically doesn't use locks at all, relying on **volatile** memory visibility to see the latest updates.
- **Weak Consistency:** Iterators reflect the state of the map at the time they were created and do *not* throw exceptions if another thread modifies the map during iteration.
- **Scalability:** By default, it supports a **concurrency level of 16**, meaning up to 16 threads can perform write operations simultaneously if they hit different buckets.

CopyOnWriteArrayList

`CopyOnWriteArrayList` is a thread-safe variant of `ArrayList` where all **mutative operations** (add, set, remove) are implemented by creating a **fresh copy** of the underlying array.

It is designed for scenarios where **reads vastly outnumber writes**, as it eliminates the need for locking during traversal.

Comprehensive Example

This example demonstrates how the "copying" mechanism allows for safe iteration even while the list is being modified.

Key Technical Aspects

- **The "Write" Process:** When you call `.add()`, the class locks a reentrant lock, copies the current array to a new one (size + 1), adds the element, and then points the internal reference to the new array.
- **Zero Locking for Reads:** Because the underlying array is never modified in place, readers don't need to worry about seeing a "half-added" element. They always see a consistent, completed version of the array.
- **Fail-Safe Iterators:** Iterators do not support `remove()`. If you try `it.remove()`, it throws `UnsupportedOperationException` because the iterator is traversing a static snapshot, not the live list.
- **Memory Overhead:** It is **not** suitable for large lists with frequent writes, as every write duplicates the entire array, which can lead to high memory consumption and GC pressure.



Quiz time

QUIZ

Which method submits tasks to ExecutorService?

2 ✗

`thread.run()`

0 ✗

`thread.wait()`

8 ✓

`executor.submit()`

0 ✗

`executor.clone()`

What does Future represent in concurrency?

11 ✓



Result of an async computation

0 ✗

Execution times

0 ✗

Thread status

0 ✗

Mutex locking

How do you check if a Future is completed?

12 ✓



0 ✗

`future.getResult()`

`future.isDone()`

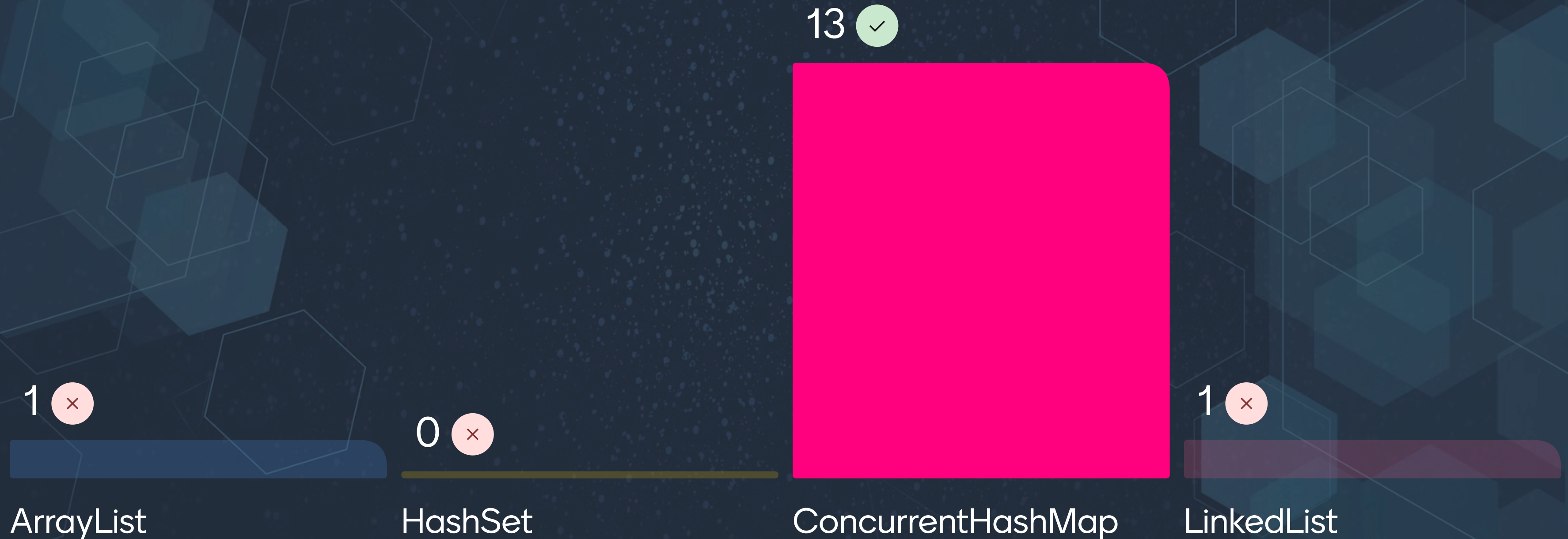
0 ✗

`future.queueStatus()`

0 ✗

`object.finalize()`

Which collection is thread-safe in Java?



How does BlockingQueue work?

12 ✓



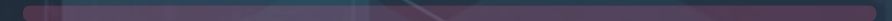
Prevents access if queue is
full/empty

2 ✗



Locks without exception

0 ✗



Executes a priority queue

0 ✗



Always overwrites data

How can multiple threads access a CopyOnWriteArrayList?

0 ✗

Use unsafe locking

2 ✗

Manually synchronize

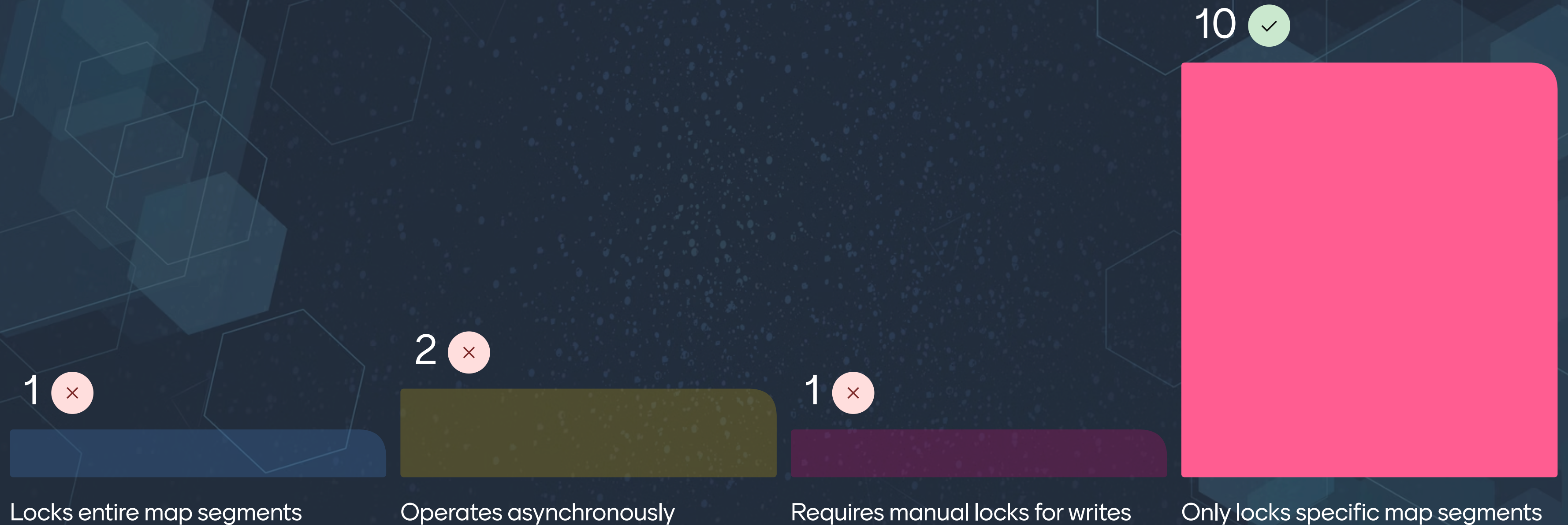
10 ✓

It is thread-safe by design

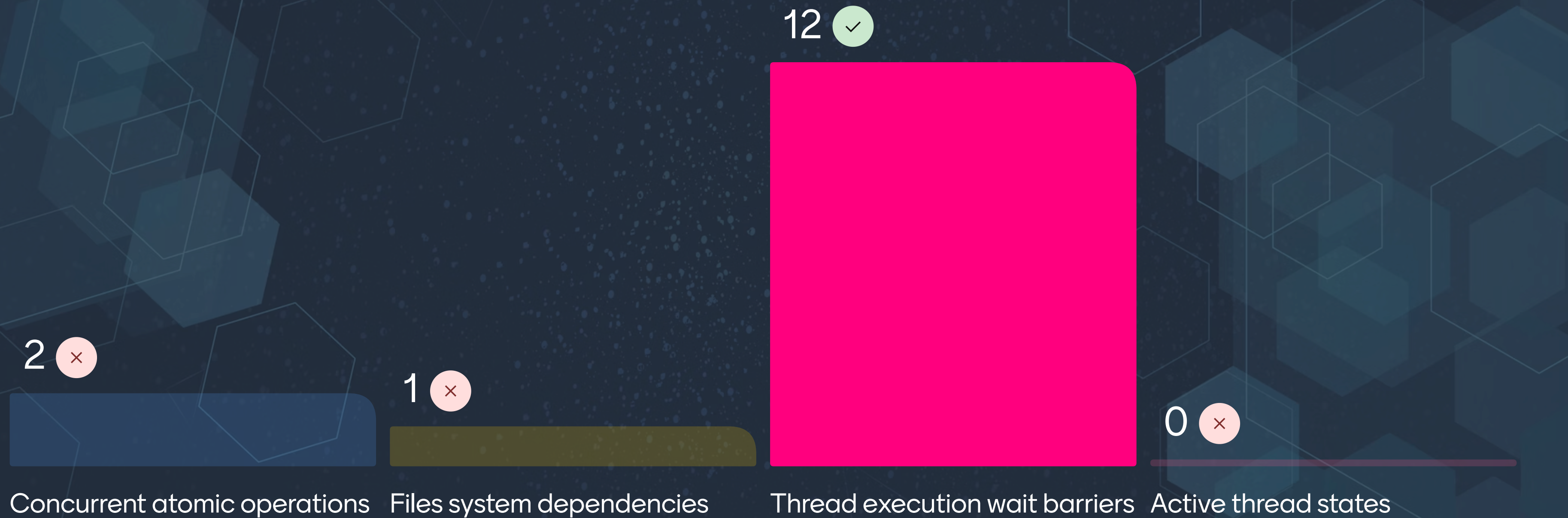
3 ✗

Execute write-only operations

What is key about ConcurrentHashMap?



What does CountdownLatch handle?



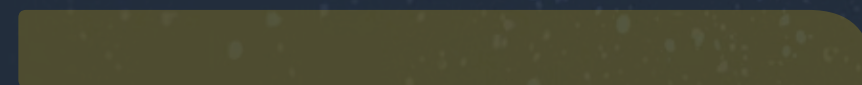
How does CyclicBarrier work?

15 ✓



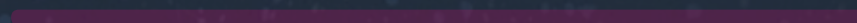
Waits for threads to reach a barrier point

1 ✗



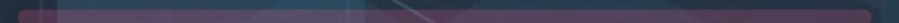
Always throws runtime errors

0 ✗



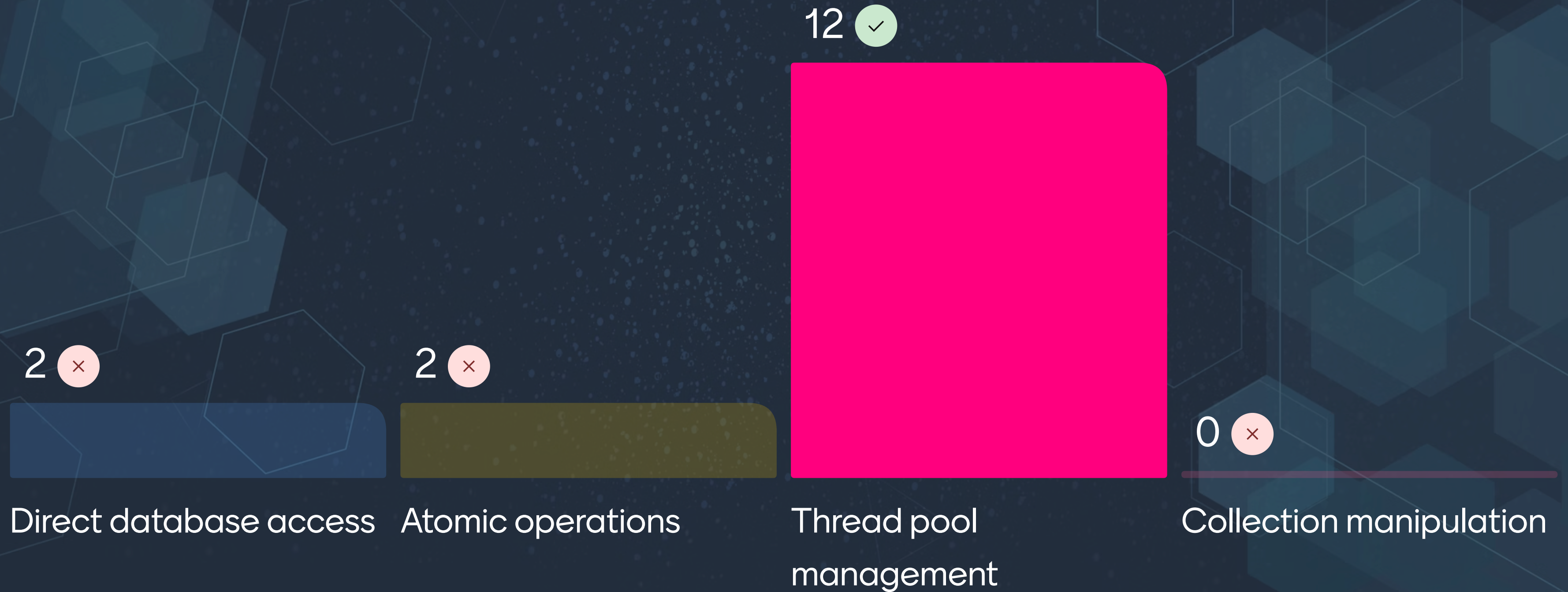
Repeatedly initializes exceptions

0 ✗

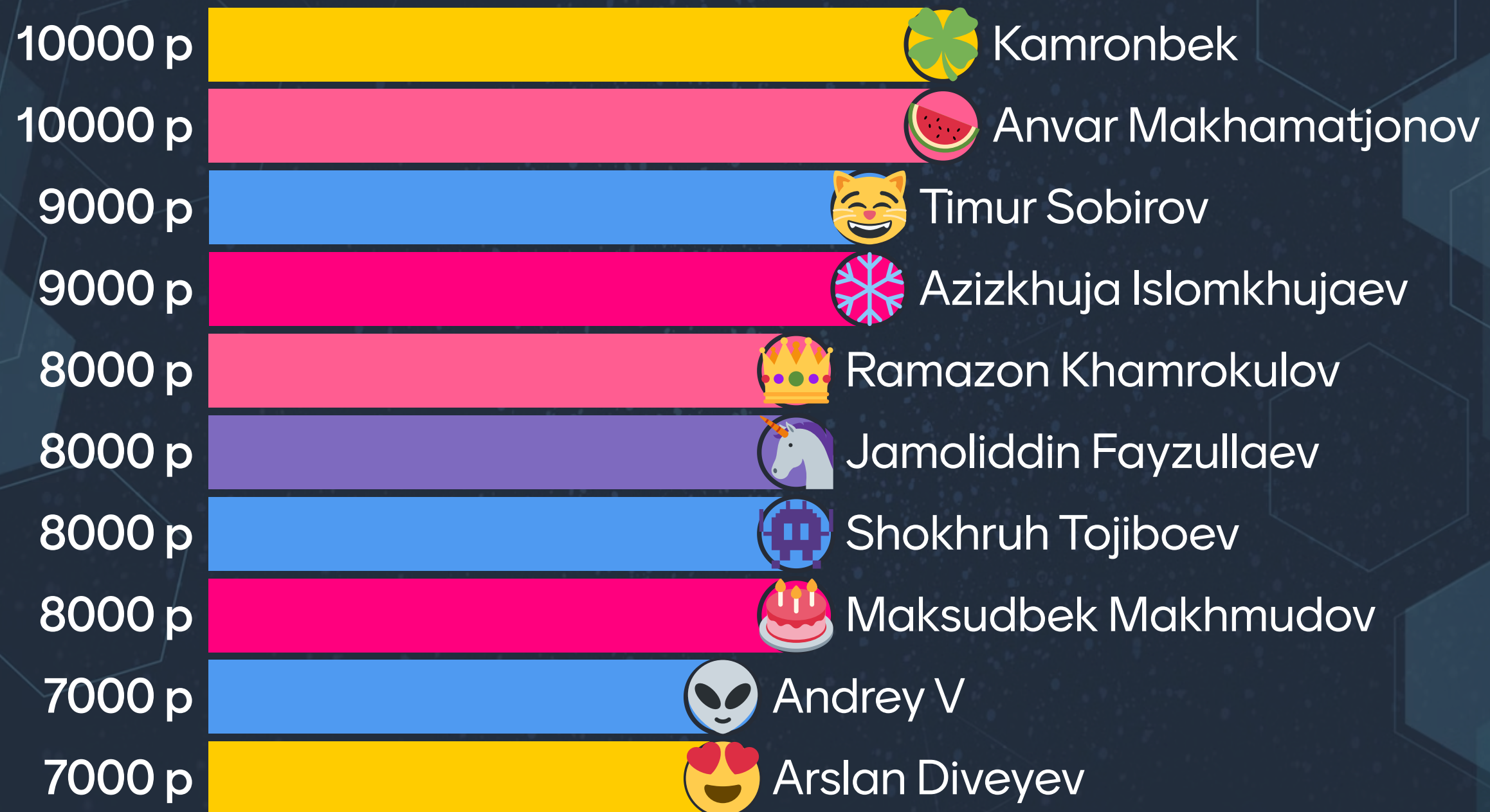


Locks without notifying threads

What does ExecutorService provide?



Quiz leaderboard



Q&A part

1 question
0 upvotes



